مستند فنی و راهنمای آموزشی جامع | نگارش پاییز ۱۴۰۵ (CLI v0.2.0)

راهنمای جامع و آموزشی تلگرام سرورلس (Telegram Serverless)

طراحی، کدنویسی، پایگاه‌داده و استقرار امن ربات‌ها و مینی‌اپ‌ها مستقیماً بر روی زیرساخت جهانی تلگرام بدون نیاز به سرور و داکر

بدون سرور، بدون داکر

پایان هزینه‌های ماهانه VPS، نگهداری سیستم‌عامل، کانفیگ Nginx، دریافت SSL و داون‌تایم سرورهای شخصی.

سندباکس ایزوله V8

اجرای فوق‌سریع در ایزوله‌های مدرن V8 در مجاورت دیتاسنترهای تلگرام با حداقل Latency ممکن.

پایگاه‌داده داخلی SQLite

ارائه دیتابیس اختصاصی ماندگار با قابلیت کوئری‌نویسی تایپ‌شده Drizzle-like و مایگریشن امن دومرحله‌ای.

میزبانی کامل Mini App

میزبانی رایگان فرانت‌اند در دامنه tgcloud.ai و تعامل با Backend از طریق RPC Endpoints اختصاصی.

نگارش: پاییز ۱۴۰۵ (بروزرسانی رسمی اکتبر ۲۰۲۶)

بستر توسعه: ابزار رسمی tgcloud CLI & Telegram Bot API

هدف سند: آموزش عملیاتی، مهندسی و مرجع توسعه‌دهندگان

فصل اول: مقدمه، فلسفه پیدایش و معماری پلتفرم

ربات‌های تلگرام ذاتاً برنامه‌هایی رویدادمحور (Event-Driven) هستند که به پیام‌ها و رویدادهای دریافتی واکنش نشان می‌دهند. در معماری سنتی، توسعه‌دهنده ناچار بود یک سرور مجازی (VPS) یا هاست اختصاصی اجاره کند، سیستم‌عامل و بسته‌های امنیتی را نگهداری نماید، فایروال و گواهی SSL تنظیم کند، وب‌سرور (نظیر Nginx یا Caddy) بالا بیاورد و یک پروسه همیشه‌روشن (با Docker، PM2 یا Systemd) را مانیتور کند تا صرفاً بتواند به یک دستور ساده `/start` پاسخ دهد.

تلگرام سرورلس (Telegram Serverless) این لایه سنگین و پرهزینه زیرساختی را به طور کامل حذف کرده است. در این فناوری، کدهای جاوااسکریپت شما مستقیماً درون زیرساخت ابری رسمی تلگرام و در محیط‌های ایزوله فوق‌سریع (Lightweight V8 Sandboxes) اجرا می‌شوند. هر بات به طور خودکار به یک پایگاه داده پایدار SQLite، کلاینت داخلی Telegram Bot API و کلاینت بهینه‌سازی‌شده HTTP دسترسی دارد؛ بدون آنکه نیازی به نصب بسته‌های خارجی یا تزریق دستی توکن‌ها و کلیدهای امنیتی باشد.

مقایسه ساختاری معماری سنتی و معماری تلگرام سرورلس:

روش سنتی (VPS / Docker)	تلگرام سرورلس (tgcloud)
هزینه ماهانه اجاره سرور	صفر هزینه نگهداری سرور
پیکربندی دستی Nginx و SSL	اجرای On-Demand در کنار Bot API
مدیریت خرابی و داون‌تایم پروسه‌ها	مقیاس‌پذیری خودکار با حجم ترافیک
تنظیم دستی آدرس Webhook	مدیریت کامل و امن وبهوک توسط تلگرام

مدل ذهنی سه‌گانه (The Mental Model)

توسعه بر بستر تلگرام سرورلس در سه نقطه اصلی جریان دارد که هماهنگی میان آن‌ها توسط ابزار خط فرمان برقرار می‌شود:

بخش	محتوا و وظیفه
پوشه محلی پروژه (Project Folder)	ماژول‌های جاوااسکریپت شامل اسکیما دیتابیس (`schema.js`)، هندلرهای رویداد، اندپوینت‌های مینی‌اپ و در صورت نیاز فایل‌های استاتیک فرانت‌اند مینی‌اپ.
ابر تلگرام (The Cloud)	نسخه مستقرشده (Deployed) ماژول‌ها، پایگاه داده SQLite پایدار ربات، و هاستینگ فایل‌های استاتیک مینی‌اپ.
ابزار خط فرمان (CLI `tgcloud`)	پل ارتباطی میان سیستم شما و ابر تلگرام؛ وظیفه نمایش تفاوت‌ها، تست محلی بدون دیپلوی، انتشار اتمیک ماژول‌ها و اعمال مایگریشن‌های امن پایگاه داده.

> 💡 **اصل کلیدی توسعه سرورلس**
>
> شما هرگز به سروری SSH نمی‌زنید. ویرایش‌ها را روی کامپیوتر خود انجام می‌دهید، با دستور `npx tgcloud run` آن را در سندباکس واقعی می‌سنجید، و با `npx tgcloud push` آن را منتشر می‌سازید. دیتابیس نیز میان تمام فراخوانی‌ها پایدار و ماندگار است.

فصل دوم: کالبدشناسی پروژه و سامانه ماژول‌ها (CLI v0.2.0+)

از نگارش 0.2.0 ابزار خط فرمان `tgcloud` (اکتبر ۲۰۲۶)، تمام ماژول‌های اجرایی ربات درون پوشه متمرکز `tgcloud/` قرار می‌گیرند. این تفکیک اجازه می‌دهد ریشه پروژه برای ابزارهای بیلد فرانت‌اند مینی‌اپ (نظیر Vite، React، Vue یا Svelte) کاملاً تمیز و استاندارد باقی بماند.

ساختار استاندارد فایل‌ها و پوشه‌ها

```
my-telegram-bot/ ├── tgcloud/ │ ├── handlers/ # هندلرهای رویداد تلگرام (هر یک فایل تخت به ازای هر Update) │ │ ├── message.js │ │ ├── callback_query.js │ │ └── inline_query.js │ ├── endpoints/ # توابع RPC که مستقیماً توسط Mini App فراخوانی می‌شوند │ │ ├── getProfile.js │ │ └── createOrder.js │ ├── lib/ # کدهای اشتراکی و هلپرها (پوشه‌بندی تودرتو مجاز است) │ │ ├── utils.js │ │ └── payments/gateway.js │ └── schema.js # فایل یکتای تعریف جداول پایگاه داده SQLite ├── dist/ # خروجی بیلد مینی‌اپ (در صورت استفاده از Mini App) ├── tgcloud.jsonc # فایل تنظیمات پروژه و هاستینگ استاتیک ├── .tgcloud/ # اطلاعات محلی CLI (قرار می‌گیرد در gitignore) و سشن ورود ├── package.json # اسکریپت‌ها و وابستگی‌های ابزارهای توسعه محلی مینی‌اپ
```

قوانین سامانه ماژول‌ها (Module System Rules)

- **محیط کاملاً ایزوله (No external npm packages):** کدهای شما در زمان اجرا فقط و فقط می‌توانند دو منبع را ایمپورت کنند: پکیج رسمی پلتفرم (`sdk`) و سایر ماژول‌های موجود در پوشه `tgcloud/` شما. دسترسی به filesystem محلی، متغیرهای محیطی سیستم، پکیج‌های نصبی node_modules و ماژول‌های پیش‌فرض Node.js (نظیر `fs` یا `crypto`) در زمان ران‌تایم وجود ندارد.
- **روش‌های ایمپورت معتبر:**
  - ایمپورت با نام ماژول (Bare Import): `import { api } from 'sdk';` یا `import { users } from 'schema';` یا `import { helper } from 'lib/utils';` .
  - ایمپورت نسبی با پسوند `.js` (پشتیبانی‌شده در CLI 0.2.0+): `import { users } from '../schema.js';` یا `import { fmt } from './format.js';` .
- **مسطح بودن پوشه‌های handlers و endpoints:** پوشه‌های `handlers/` و `endpoints/` باید کاملاً تخت (Flat) باشند و هیچ ساب‌فولدری در آن‌ها مجاز نیست. در مقابل، پوشه `lib/` از ساختارهای تودرتوی دلخواه پشتیبانی می‌کند.

> ⚠️ **شرط تحویل آپدیت‌ها در تلگرام**
>
> تلگرام تنها آپدیت‌هایی را به سمت ربات شما ارسال می‌کند که فایل مربوط به آن‌ها در پوشه `handlers/` وجود داشته و **غیرخالی** باشد. اگر فایل `handlers/callback_query.js` در پروژه شما وجود نداشته باشد، پلتفرم هیچ‌گونه ترافیک مربوط به دکمه‌های شیشه‌ای را تحویل نمی‌گیرد و هیچ کدی برای آن اجرا نخواهد شد.

فصل سوم: راه‌اندازی سریع و چرخه توسعه (CLI Workflow)

برای راه‌اندازی اولین ربات سرورلس، تنها به Node.js نسخه ۱۸ یا بالاتر و یک حساب تلگرام نیاز دارید.

گام ۱: فعال‌سازی Serverless در BotFather

1. ربات **@BotFather** را در تلگرام باز کنید.
2. ربات خود را انتخاب کرده و وارد منوی **Serverless** شوید.
3. گزینه فعال‌سازی را بزنید تا قابلیت سرورلس برای ربات شما روشن شود.

4. از بخش **CLI Access > Access token**، توکن دسترسی ابزار خط فرمان را کپی کنید. فرمت این توکن به شکل `app<id>:<secret>` است.

> **تفاوت حیاتی دو توکن تلگرام**
>
> توکن دسترسی CLI (شروع با `app...`) با توکن کلاسیک Bot API (شروع با ارقام، مانند `123456:ABC-DEF...`) کاملاً متفاوت است. توکن CLI برای مدیریت، آپلود کد و دیتابیس است؛ در حالی که ران‌تایم سرورلس نیازی به وارد کردن توکن Bot API ندارد و خودبه‌خود احراز هویت شده است.

گام ۲: ایجاد پروژه و ورود به حساب

```
# ایجاد پروژه جدید با اسکلت رسمی تلگرام
npm create @tgcloud/bot my_serverless_bot
cd my_serverless_bot

# اتصال پروژه به ربات تلگرام (تنها یکبار نیاز است)
npx tgcloud login
```

دستور `login` توکن دسترسی شما را دریافت و آن را در فایل محلی `.tgcloud/credentials` ذخیره می‌کند. در محیط‌های Continuous Integration (CI/CD)، می‌توانید این توکن را مستقیماً در متغیر محیطی `TGCLOUD_TOKEN` قرار دهید تا نیازی به ورود دستی نباشد.

گام ۳: تست محلی با دستور run (بدون نیاز به دیپلوی)

دستور `tgcloud run` منطق کدها را بر روی سندباکس واقعی پلتفرم پیش از انتشار می‌سنجد:

```
# تست مستقیم هندلر پیام با ارسال پی‌لود به فرمت JSON5
npx tgcloud run handlers/message '{ chat: { id: 1001 }, text: "/start" }'
```

خروجی کنسول (شامل `console.log` ها)، مقدار بازگشتی تابع و مدت‌زمان اجرا (Latency) فوراً در ترمینال شما نمایش داده می‌شود.

گام ۴: چرخه انتشار (Push)

```
npx tgcloud push
```

اکنون ربات شما به صورت زنده بر روی زیرساخت تلگرام فعال است! کافی است به ربات در تلگرام پیام بدهید تا پاسخ را دریافت کنید.

فصل چهارم: پایگاه‌داده داخلی SQLite و معماری مایگریشن

هر ربات دارای یک پایگاه داده پایدار SQLite اختصاصی است. تعریف جداول درون فایل `tgcloud/schema.js` و با استفاده از یک DSL تایپ‌شده و مدرن کاملاً شبیه به Drizzle ORM صورت می‌گیرد.

تعریف جداول در `schema.js`

```
import { table, integer, text, boolean, real, json, index, sql } from 'sdk/db';

// تعریف جدول کاربران
export const users = table('users', {
  id:        integer('id').primaryKey({ autoIncrement: true }),
  tgId:      integer('tg_id').unique().notNull(),
  fullName:  text('full_name').notNull(),
  score:     integer('score').default(0),
  balance:   real('balance').default(0.0),
  isActive:  boolean('is_active').default(true),
  metadata:  json('metadata'),
  createdAt: integer('created_at', { mode: 'timestamp' }).default(sql`(unixepoch())`),

  // نشانه‌گذاری فیلد منسوخ‌شده برای حذف ایمن در مایگریشن بعدی:
  oldNotes:  text('old_notes').deprecated('replaced by metadata'),
}, (t) => ({
  tgIdIdx:   index('idx_users_tg_id').on(t.tgId),
  activeIdx: index('idx_active_users').on(t.createdAt).where(sql`is_active = 1`),
}));
```

🚫 قانون حیاتی: فقدان کلید خارجی (No Foreign Keys)

سندباکس دیتابیس تلگرام همواره با تنظیم `PRAGMA foreign_keys = off` اجرا می‌شود. اگر در کدهای اسکیما از دستوراتی نظیر `.references()` یا `foreignKey()` استفاده کنید، کد شما با خطای زمان کامپایل متوقف شده و دیپلوی رد خواهد شد! روابط باید از طریق ستون‌های عادی (مانند `userId: integer('user_id')`) مدل شده و یکپارچگی ارجاعی در منطق برنامه حفظ شود.

کوئری‌نویسی با آبجکت `db`

آبجکت `db` یک Query Builder زنجیره‌ای و روان است. به یاد داشته باشید که **تمام کوئری‌ها ناهمگام (Async) هستند و همواره باید `await` شوند:**

```
import { db } from 'sdk';
import { users } from 'schema';
import { eq, and, gt, desc, sql } from 'sdk/db';

// ۱. انتخاب اطلاعات (Select)
const allUsers = await db.select().from(users).all();
const user = await db.select().from(users).where(eq(users.tgId, 123456)).get();

// ۲. درج و به‌روزرسانی در صورت وجود (Upsert با returning)
const [savedUser] = await db.insert(users)
  .values({ tgId: 123456, fullName: 'مهدی', score: 10 })
  .onConflictDoUpdate({
    target: users.tgId,
    set: { score: sql`${users.score} + 10` }
  })
  .returning()
  .run();

// ۳. شمارش رکوردها با هلپر پرسرعت $count
const activeCount = await db.$count(users, eq(users.isActive, true));

// ۴. حذف و به‌روزرسانی شرطی
await db.update(users).set({ score: 0 }).where(eq(users.tgId, 123456)).run();
await db.delete(users).where(eq(users.isActive, false)).run();
```

معماری تفکیک دیپلوی و مایگریشن (Two-Step Model)

یکی از مهم‌ترین تصمیمات معماری در تلگرام سرورلس، **جداسازی قطعی انتشار کد از تغییر دیتابیس** است. دستور `tgcloud push` هرگز دیتابیس را لمس نمی‌کند تا از فاجعه حذف ناخواسته داده‌ها جلوگیری شود:

1. با اجرای `npx tgcloud push`، فایل `schema.js` جدید آپلود شده و وضعیت مغایرت به شما گزارش می‌شود.
2. سپس دستور `npx tgcloud migrate` را اجرا می‌کنید تا لیست تغییرات را بررسی و تایید نمایید.

سطح تغییر	ماهیت تغییر	نحوه اعمال در migrate
safe	تغییرات افزایشی (جدول جدید، ستون جدید، ایندکس جدید)	پس از یک تایید کلی، همگی به صورت یکجا و آنی اعمال می‌شوند.
warning	تغییرات تخریبی یا پرهزینه (حذف ستون، حذف جدول)	تک‌به‌تک نشان داده شده و برای هرکدام تایید مجزا درخواست می‌شود.
manual	تغییرات ناسازگار (نظیر تغییر نوع ستون)	به صورت خودکار اعمال نمی‌شود؛ باید با اسکریپت SQL دستی منتقل شود.

فصل پنجم: هندلرهای رویداد و کتابخانه SDK

هندلرها درگاه‌های اصلی ورود پیام و رویداد هستند. نام هر فایل در پوشه `tgcloud/handlers/` متناظر با یک فیلد از شیء **Update** در تلگرام است (نظیر `message.js`، `callback_query.js`، `inline_query.js`، `chat_member.js`).

ساختار هندلر و پارامترها

```
// tgcloud/handlers/message.js
import { api, db } from 'sdk';
import { users } from 'schema';
import { eq } from 'sdk/db';

export default async function (payload, ctx) {
  // payload حاوی شیء Message است بازشده (همان update.message)
  const chatId = payload.chat.id;
  const text = payload.text;

  // ctx آرگومان دوم است حاوی کل آبجکت خام تلگرام (شامل update_id)
  const updateId = ctx.update.update_id;

  await api.sendMessage({
    chat_id: chatId,
    text: `پیام شما دریافت شد: ${text ?? '(بدون متن)'}`,
  });
}
```

تعامل با Telegram Bot API از طریق `api`

- **خروجی مستقیماً باز شده است (Unwrapped):** در Bot API کلاسیک، پاسخ‌ها در یک پوشش `{ ok: true, result: ... }` بازگردانده می‌شدند. در ماژول `api` تلگرام سرورلس، حاصل فراخوانی مستقیماً همان مقدار `result` است.
- **مدیریت خطا با `BotApiError`:** هر زمان تلگرام خطایی برگرداند، کتابخانه یک استثنا از جنس `BotApiError` پرتاب می‌کند:

```
import { api, BotApiError } from 'sdk';

try {
  await api.deleteMessage({ chat_id, message_id });
} catch (err) {
  if (err instanceof BotApiError && err.code === 400) {
    // پیام از قبل حذف شده بوده است؛ خطا را نادیده می‌گیریم
  } else {
    throw err;
  }
}
```

ارسال و دریافت فایل‌ها (Files)

برای آپلود فایل‌ها، از شیء `InputFile` استفاده می‌شود که بایت‌ها را به شکل مستقیم و چندبخشی (Multipart Stream) تا سقف **۵۰ مگابایت** ارسال می‌کند:

```
import { api, InputFile } from 'sdk';

// آپلود فایل سندی با بایت‌های خام Uint8Array
const doc = new InputFile(pdfBytes, 'report.pdf', { type: 'application/pdf' });
await api.sendDocument({ chat_id: chatId, document: doc });

// دانلود فایل تا سقف ۲۰ مگابایت با شناسه file_id
const downloadedBytes = await api.getFileContent(fileId);
```

فراخوانی وب‌سرویس‌های خارجی و هوش مصنوعی با `fetch`

کلاینت `fetch` برای تعامل با APIهای خارجی، درگاه‌های پرداخت، وب‌هوک‌ها و مدل‌های LLM در دسترس است (حداکثر حجم پاسخ: ۳۰ مگابایت):

```
import { fetch } from 'sdk';

const res = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer sk-...' },
  body: fetch.body.json({
    model: 'gpt-4o',
    messages: [{ role: 'user', content: 'سلام' }]
  })
});
const data = await res.json();
```

فصل ششم: توسعه بک‌اند و میزبانی Mini Appها

پلتفرم سرورلس علاوه بر ربات‌ها، بستری کامل برای اجرای مینی‌اپ‌های تلگرام (Telegram Mini Apps) است. شما می‌توانید فرانت‌اند خود را روی دامنه پرسرعت و امن `https://app<app_id>.tgcloud.ai/` میزبانی کرده و ارتباط میان فرانت‌اند و دیتابیس را با **Endpoints** برقرار کنید.

تعریف اندپوینت‌های RPC در `tgcloud/endpoints/`

اندپوینت‌ها توابعی هستند که فرانت‌اند مینی‌اپ آن‌ها را فراخوانی می‌کند. بزرگ‌ترین مزیت این اندپوینت‌ها، **احراز هویت ذاتی و غیرقابل جعل** آن‌ها توسط تلگرام است:

```
// tgcloud/endpoints/getProfile.js
import { db, EndpointError } from 'sdk';
import { users } from '../schema.js';
import { eq } from 'sdk/db';

export default async function (input, ctx) {
  // ctx.initData توسط تلگرام به صورت رمزنگاری‌شده اعتبارسنجی شده است!
  const user = ctx.initData.user;
  if (!user) {
    throw new EndpointError('کاربر نامعتبر است', { code: 'UNAUTHORIZED' });
  }

  const record = await db.select().from(users).where(eq(users.tgId, user.id)).get();
  if (!record) {
    throw new EndpointError('حساب کاربری یافت نشد', { code: 'NOT_FOUND' });
  }

  return { profile: record, serverTime: Date.now() };
}
```

> **🔒 امنیت خودکار بدون نیاز به JWT یا هش‌کردن دستی initData**
>
> در روش سنتی، بک‌اند باید رشته `initData` را با توکن ربات به صورت HMAC-SHA256 هش می‌کرد تا از جعلی نبودن کاربر مطمئن شود. در تلگرام سرورلس، این کار مستقیماً توسط زیرساخت تلگرام انجام می‌شود و درخواست‌های فاقد اعتبار پیش از رسیدن به کد شما با خطای 401 مسدود می‌شوند!

فراخوانی اندپوینت از فرانت‌اند مینی‌اپ

در کد جاوااسکریپت مینی‌اپ با اسکریپت رسمی `telegram-web-app.js` :

```
// در فرانت‌اند مینی‌اپ:
Telegram.WebApp.Serverless.call('getProfile', { lang: 'fa' }, function (err, res) {
  if (err) {
    if (err.type === 'ENDPOINT_ERROR') {
      alert('خطا: ' + err.message + ' (کد: ' + err.parameters.code + ')');
    } else {
      console.error('خطای سیستم:', err.type);
    }
    return;
  }
  console.log('اطلاعات پروفایل دریافت شد:', res.profile);
});
```

پیکربندی میزبانی استاتیک در `tgcloud.jsonc`

برای استقرار همزمان فرانت‌اند در کنار ماژول‌های ربات:

```
{
  "$schema": "./node_modules/@tgcloud/cli/schema/tgcloud.json",
  "static": {
    "source": "dist",
    "spa": true,
    "immutable": ["/assets/*"],
    "headers": [{ "path": "/*", "set": { "Referrer-Policy": "no-referrer" } }],
    "redirects": [{ "from": "/old", "to": "/new", "status": 301 }]
  }
}
```

فصل هفتم: مرجع کامل خط فرمان (`tgcloud` CLI Reference)

ابزار خط فرمان `tgcloud` تمام جنبه‌های تست، بیلد، بررسی مغایرت و استقرار را پوشش می‌دهد:

دستور (Command)	کاربرد و جزئیات اجرایی
`npx tgcloud init`	اسکلت‌بندی پروژه سرورلس جدید شامل پوشه `tgcloud/`، فایل کانفیگ `tgcloud.jsonc` و اسناد SDK.
`npx tgcloud login`	اتصال پروژه محلی به ربات تلگرام با دریافت توکن دسترسی CLI از BotFather.
`npx tgcloud add <target>`	ساخت خودکار یک ماژول آماده (مانند `handlers/callback_query` یا `endpoints/checkout`).
`npx tgcloud status`	مشاهده آفلاین وضعیت فایل‌های محلی در مقایسه با نسخه دیپلوی‌شده در کلاود.
`npx tgcloud diff`	نمایش خط به خط تغییرات اعمال‌شده در ماژول‌ها نسبت به نسخه ابری.
`npx tgcloud run <mod> [args]`	اجرای فوری ماژول روی سندباکس ابری با کدهای محلی (بدون نیاز به دیپلوی) با ورودی JSON5.
`npx tgcloud push [files]`	استقرار اتمیک ماژول‌ها و فایل‌های استاتیک مینی‌اپ در سرور تلگرام.
`npx tgcloud migrate`	محاسبه تفاوت اسکیما با دیتابیس زنده و اعمال تعاملی تغییرات و مایگریشن‌ها.
`npx tgcloud webhook sync`	بررسی سلامت وبهوک و همگام‌سازی خودکار لیست `allowed_updates` با هندلرهای موجود.
`npx tgcloud fetch`	دریافت آخرین اطلاعات از وضعیت کلاود بدون دستکاری در فایل‌های محلی.
`npx tgcloud pull`	دانلود کامل آخرین نسخه مستقر در کلاود و جایگزینی فایل‌های کاری محلی.
`npx tgcloud reset`	لغو تغییرات محلی و بازگردانی ماژول‌ها به آخرین وضعیت مستقر در کلاود.
`npx tgcloud upgrade`	انتقال خودکار ساختار پروژه‌های قدیمی (پیش از نگارش 0.2.0) به داخل پوشه `tgcloud/`.

همگام‌سازی وبهوک با `webhook sync`

```
# بررسی وضعیت کنونی وبهوک، تاخیرها و خطاهای احتمالی
npx tgcloud webhook

# همگام‌سازی خودکار و پاک‌سازی آپدیت‌های انباشته‌شده قدیمی در صف تلگرام
npx tgcloud webhook sync --drop-pending
```

مدیریت کار تیمی و تضاد نسخه‌ها (Revisions)

پروژه در ابر تلگرام دارای یک شماره نگارش (Revision) افزایشی است. اگر هم‌تیمی شما تغییری را دیپلوی کرده باشد و شما سعی در `push` داشته باشید، پلتفرم از رونویسی ناخواسته کار دیگران جلوگیری می‌کند. در این شرایط ابتدا با `tgcloud fetch` یا `tgcloud pull` هماهنگ شوید و سپس تغییرات خود را مجدداً `push` کنید.

فصل هشتم: سناریوی کاربردی جامع (Full-Stack Architecture Example)

برای درک چگونگی اتصال تمام قطعات پازل، سناریوی واقعی یک ربات فروشگاهی به همراه مینی‌اپ و سبد خرید را در ادامه پیاده‌سازی می‌کنیم:

۱. ساختار پایگاه داده در `tgcloud/schema.js`

```
import { table, integer, text, real, sql } from 'sdk/db';

export const products = table('products', {
  id:    integer('id').primaryKey({ autoIncrement: true }),
  title: text('title').notNull(),
  price: real('price').notNull(),
  stock: integer('stock').default(0),
});

export const orders = table('orders', {
  id:        integer('id').primaryKey({ autoIncrement: true }),
  buyerTgId: integer('buyer_tg_id').notNull(),
  productId: integer('product_id').notNull(), // ارجاع نرم‌افزاری بدون foreign key
  createdAt: integer('created_at').default(sql`(unixepoch())`),
});
```

۲. هندلر شروع و ارسال دکمه مینی‌اپ در `tgcloud/handlers/message.js`

```
import { api } from 'sdk';

export default async function (message) {
  if (message.text === '/start') {
    await api.sendMessage({
      chat_id: message.chat.id,
      text: 'به فروشگاه خوش آمدید! برای مشاهده محصولات، روی دکمه زیر کلیک کنید:',
      reply_markup: {
        inline_keyboard: [[
          { text: '🛒 باز کردن فروشگاه', web_app: { url: 'https://app1234.tgcloud.ai/' } }
        ]]
      }
    });
  }
}
```

۳. اندپوینت خرید امن درون مینی‌اپ در `tgcloud/endpoints/buyProduct.js`

```
import { db, api, EndpointError } from 'sdk';
import { products, orders } from '../schema.js';
import { eq, gt, sql } from 'sdk/db';

export default async function (input, ctx) {
  const user = ctx.initData.user;
  const productId = input.productId;

  // ۱. بررسی موجودی کالا
  const item = await db.select().from(products).where(eq(products.id, productId)).get();
  if (!item || item.stock < 1) {
    throw new EndpointError('کالای مورد نظر ناموجود است', { code: 'OUT_OF_STOCK' });
  }

  // ۲. کسر موجودی و ثبت سفارش
  await db.update(products).set({ stock: sql`${products.stock} - 1` }).where(eq(products.id, productId
  await db.insert(orders).values({ buyerTgId: user.id, productId }).run();

  // ۳. ارسال پیام تایید در چت کاربر از طریق Bot API
  await api.sendMessage({
    chat_id: user.id,
    text: `سفارش شما برای "${item.title}" با موفقیت ثبت شد.`,
  });

  return { success: true, orderId: Date.now() };
}
```

فصل نهم: اشتباهات رایج، چک‌لیست استقرار و الگوهای ضدطراحی

برای اطمینان از عملکرد پایدار بات و مینی‌اپ، جدول خطاهای متداول و چک‌لیست نهایی را همیشه به خاطر بسپارید:

اشتباه متداول (Anti-Pattern)	پیامد و علت خطا	راهکار صحیح و استاندارد
نصب پکیج‌های npm برای ران‌تایم (مانند axios یا lodash)	سندباکس به node_modules دسترسی ندارد و خطا می‌دهد.	تنها از پکیج درونی `sdk` و ماژول `fetch` استفاده کنید.
تعریف `.references()` در اسکیما برای Foreign Key	پلتفرم با پرتاب استثنا مانع دیپلوی می‌شود.	کلید را به عنوان integer تعریف کرده و یکپارچگی را در کد حفظ کنید.
فراموش کردن `tgcloud migrate` بعد از تغییر اسکیما	جداول دیتابیس آپدیت نمی‌شوند و کدهای جدید خطا می‌خورند.	همیشه بعد از `push` اسکیما، دستور `migrate` را اجرا کنید.
عدم قرار دادن `await` پشت متدهای `db`	پرامیس حل‌نشده باقی مانده و دیتابیس خوانده یا نوشته نمی‌شود.	همیشه پشت `db.select()`، `insert()`، `update()` کلمه `await` بگذارید.
تنظیم دستی Webhook با متد `setWebhook`	وبهوک سرورلس دچار اختلال و خرابی مسیر می‌شود.	از دستور رسمی `npx tgcloud webhook sync` استفاده کنید.
قراردادن پوشه تودرتو داخل `handlers/` یا `endpoints/`	پلتفرم فایل‌های درون پوشه‌های تودرتو را به عنوان رویداد شناسایی نمی‌کند.	این دو پوشه باید کاملاً مسطح (Flat) باشند؛ کدهای اشتراکی را به `lib/` ببرید.

چک‌لیست استقرار نهایی (Zero-Downtime Deployment Checklist)

📋 مراحل تایید نهایی پیش از لانچ ربات به کاربران عمومی:

1. تست تک‌تک هندلرها و اندپوینت‌ها به صورت آفلاین با `npx tgcloud run <module>`.
2. بررسی عدم وجود خطای نگارشی در `tgcloud/schema.js` و اطمینان از عدم استفاده از Foreign Key.
3. بیلد فرانت‌اند مینی‌اپ در صورت وجود با `npm run build`.
4. انتشار اتمیک کدهای جدید با `npx tgcloud push`.
5. اعمال تغییرات دیتابیس با `npx tgcloud migrate` و تایید تغییرات.
6. اطمینان از سلامت وبهوک و لیست آپدیت‌های مجاز با `npx tgcloud webhook sync`.